A27. API REST con patrón MVC

Queremos desarrollar un backend para un ayuntamiento que cuente con un servicio de alquiler de bicicletas. Vamos a desarrollar la API de un proyecto siguiendo la estructura que express-generator nos proporcionó en la tarea anterior.

Ejercicio 1. Modelo Bicicleta

En la estructura de proyecto generada en la práctica anterior, en la carpeta raíz crearemos una carpeta llamada models y, dentro de ella, un archivo Bicicleta.js que contendrá toda la lógica del modelo Bicicleta. Es una buena práctica escribir la primera letra de los modelos en mayúscula y nombrarlos en singular.

A continuación, crea un constructor para el modelo con sus atributos:

```
let Bicicleta = function (id, color, modelo, ubicacion) {
   this.id = id;
   this.color = color;
   this.modelo = modelo;
   this.ubicacion = ubicacion;
}
```

Crea un array llamado allBicis que contendrá objetos de tipo Bicicleta. Cuando utilicemos MongoDB para almacenar los modelos, no necesitaremos este arreglo. Por ahora, lo usaremos para simplificar::

```
Bicicleta.allBicis = [];
```

Crea un método para añadir una bicicleta al final del array:

```
Bicicleta.add = function (bici) {
   this.allBicis.push(bici);
}
```

Agreguemos un par de bicicletas al array de bicicletas en memoria. Estos datos están “hardcodeados”, lo cual no es la mejor opción, pero lo hacemos así para simplificar.

```
let a = new Bicicleta(1, "Rojo", "Trek", [28.503789, -13.853296]);
let b = new Bicicleta(2, "Azul", "Orbea", [28.501367, -13.853476]);
Bicicleta.add(a);
```

Bicicleta.add(b);

Exporta el módulo para que sea accesible en toda tu app.

module.exports = Bicicleta;

```
models > JS Bicicleta.js > ...
1   let Bicicleta = function (id, color, modelo, ubicacion) {
2       this.id = id;
3       this.color = color;
4       this.modelo = modelo;
5       this.ubicacion = ubicacion;
6   }
7
8   Bicicleta.allBicis = [];
9
10  Bicicleta.add = function (bici) {
11      this.allBicis.push(bici);
12  }
13
14
15  let a = new Bicicleta(1, "Rojo", "Trek", [28.503789, -13.853296]);
16  let b = new Bicicleta(2, "Azul", "Orbea", [28.501367, -13.853476]);
17  Bicicleta.add(a);
18  Bicicleta.add(b);
19
20  module.exports = Bicicleta;
21
```

Ejercicio 2. El controller

Creamos una carpeta api dentro de la carpeta controllers y, dentro de ella, un archivo BicicletaControllerAPI.js:

```
let Bicicleta = require("../../models/Bicicleta");

exports.bicicleta_list = function(req, res) {
   res.status(200).json ({
      bicicletas: Bicicleta.allBicis
   });
};
```

Ahora tenemos que crear la ruta para ese controlador. Creamos una carpeta api dentro de routes y, dentro de ella, un archivo bicicletas.js:

```
let express = require('express');
let router = express.Router();
let BicicletaControllerAPI = require("../../controllers/api/BicicletaControllerAPI");

router.get("/", BicicletaControllerAPI.bicicleta_list);

module.exports = router;
```

Tenemos que modificar app.js para agregar el router que se encargará de la API (coloca el código en las líneas correspondientes):

```
var bicicletasAPIRouter = require('./routes/api/bicicletas');
...

//If there are several versions of the API, /api/v1/bicicletas is usually used
app.use('/api/bicicletas', bicicletasAPIRouter);
```

Para probar lo que acabamos de hacer, utilizamos Postman o cURL y probamos con el método HTTP GET.

BicicletaControllerAPI.js

```
let Bicicleta = require("../../models/Bicicleta");

exports.bicicleta_list = function(req, res) {
    res.status(200).json ({
        bicicletas: Bicicleta.allBicis
    });
};
```

app.js

```
JS app.js > ...
var createError = require('http-errors');
var express = require('express');
var path = require('path');
var cookieParser = require('cookie-parser');
var logger = require('morgan');

var indexRouter = require('./routes/index');
var usersRouter = require('./routes/users');

var bicicletasAPIRouter = require('./routes/api/bicicletas');

var app = express();

// view engine setup
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'pug');

app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/', indexRouter);
app.use('/users', usersRouter);

app.use('/api/bicicletas', bicicletasAPIRouter);

// catch 404 and forward to error handler
app.use(function(req, res, next) {
  next(createError(404));
});

// error handler
app.use(function(err, req, res, next) {
  // set locals, only providing error in development
  res.locals.message = err.message;
  res.locals.error = req.app.get('env') === 'development' ? err : {};

  // render the error page
  res.status(err.status || 500);
  res.render('error');
});

module.exports = app;
```

bicicletas.js

```
JS bicicletas.js ×
routes > api > JS bicicletas.js > ...
1  let express = require('express');
2  let router = express.Router();
3  let BicicletaControllerAPI = require("../../controllers/api/BicicletaControllerAPI");
4
5  router.get("/", BicicletaControllerAPI.bicicleta_list);
6
7  module.exports = router;
8
```

```
GET  http://localhost:3000/api/bicicletas                                   Send
Params  Authorization  Headers (9)  Body ●  Scripts  Settings               Code  Cookies
none  form-data  x-www-form-urlencoded  raw  binary  GraphQL  JSON          Beautify
1 {
2 "id":5,
3 "color": "rojo",
4 "modelo": "urbano",
5 "latitud": 28.510288,
6 "longitud": -13.861913
7 }

Body  Cookies  Headers (7)  Test Results                Status: 200 OK  Time: 15 ms  Size: 403 B
Pretty  Raw  Preview  JSON
1 {"bicicletas":[{"id":1,"color":"Rojo","modelo":"Trek","ubicacion":[28.503789,-13.853296]},{"id":2,"color":"Azul","modelo":"Orbea","ubicacion":[28.501367,-13.853476]}]}
```

Ejercicio 3. POST

Ahora implementemos el método POST. Vamos al controlador de la API y exportamos el método bicicleta_create:

```
exports.bicicleta_create = function(req,res){
   let bici = new Bicicleta(req.body.id, req.body.color, req.body.modelo);
   bici.ubicacion = [req.body.latitud, req.body.longitud];

   Bicicleta.add(bici);

   res.status(201).json({
      bicicleta: bici
   })
}
```

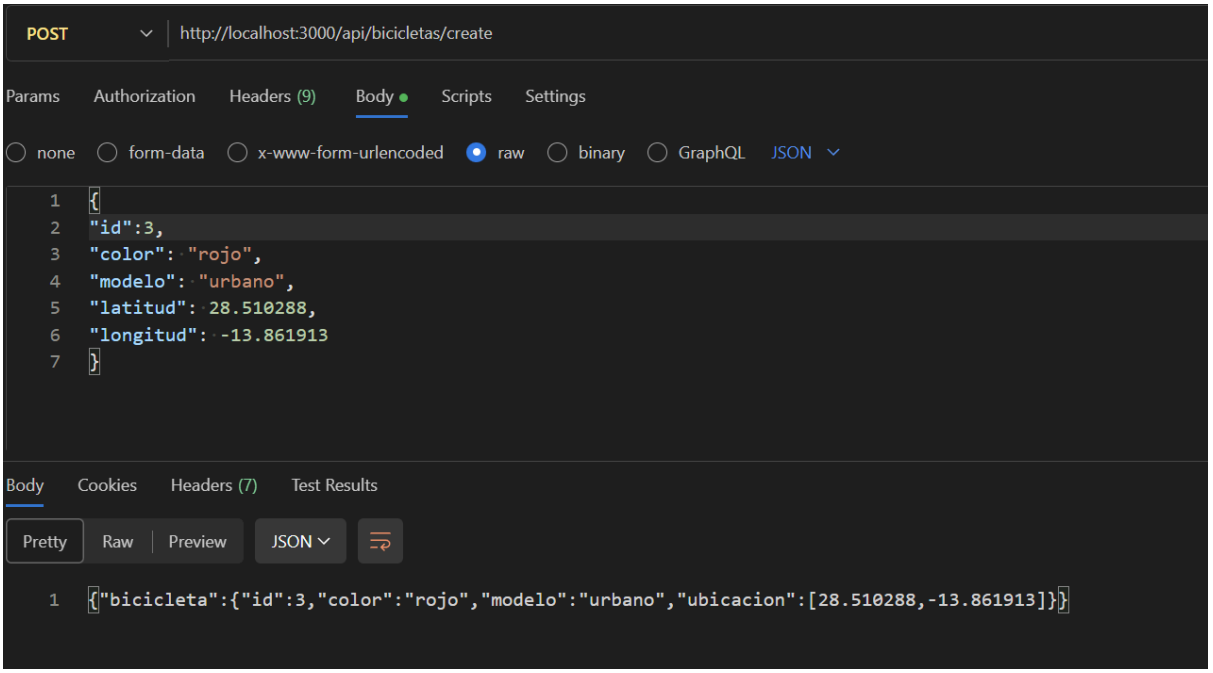
Ahora tenemos que añadir la ruta:

```
router.post("/create", BicicletaControllerAPI.bicicleta_create);
```

Añade una nueva bicicleta usando Postman.

Tendrás que obtener el código 201 y el JSON como respuesta.

Haz un GET con Postman y verifica que la bicicleta se haya agregado.



Ejercicio 4. DELETE

Vamos a implementar el método DELETE:

```
exports.bicicleta_delete = function(req,res){
   Bicicleta.removeById(req.body.id);
   res.status(204).send();
};
```

Tendrás que implementar el método removeById en el modelo Bicicleta.

La ruta será:

```
router.delete("/delete", BicicletaControllerAPI.bicicleta_delete);
```

Ten en cuenta que debes obtener el código 204, que es una respuesta correcta pero sin contenido.

models > JS Bicicleta.js > removeById

```js
let Bicicleta = function (id, color, modelo, ubicacion) {
    this.id = id;
    this.color = color;
    this.modelo = modelo;
    this.ubicacion = ubicacion;
}

Bicicleta.allBicis = [];

Bicicleta.add = function (bici) {
    this.allBicis.push(bici);
}

Bicicleta.removeById = function (id) {
    const index = this.allBicis.findIndex(bici => bici.id == id);
    if (index >= 0) {
        this.allBicis.splice(index, 1);
    }
}

let a = new Bicicleta(1, "Rojo", "Trek", [28.503789, -13.853296]);
let b = new Bicicleta(2, "Azul", "Orbea", [28.501367, -13.853476]);
Bicicleta.add(a);
Bicicleta.add(b);

module.exports = Bicicleta;
```

DELETE http://localhost:3000/api/bicicletas/delete

Params Authorization Headers (9) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
"id":3
}
```

Body Cookies Headers (4) Test Results

Pretty Raw Preview Text

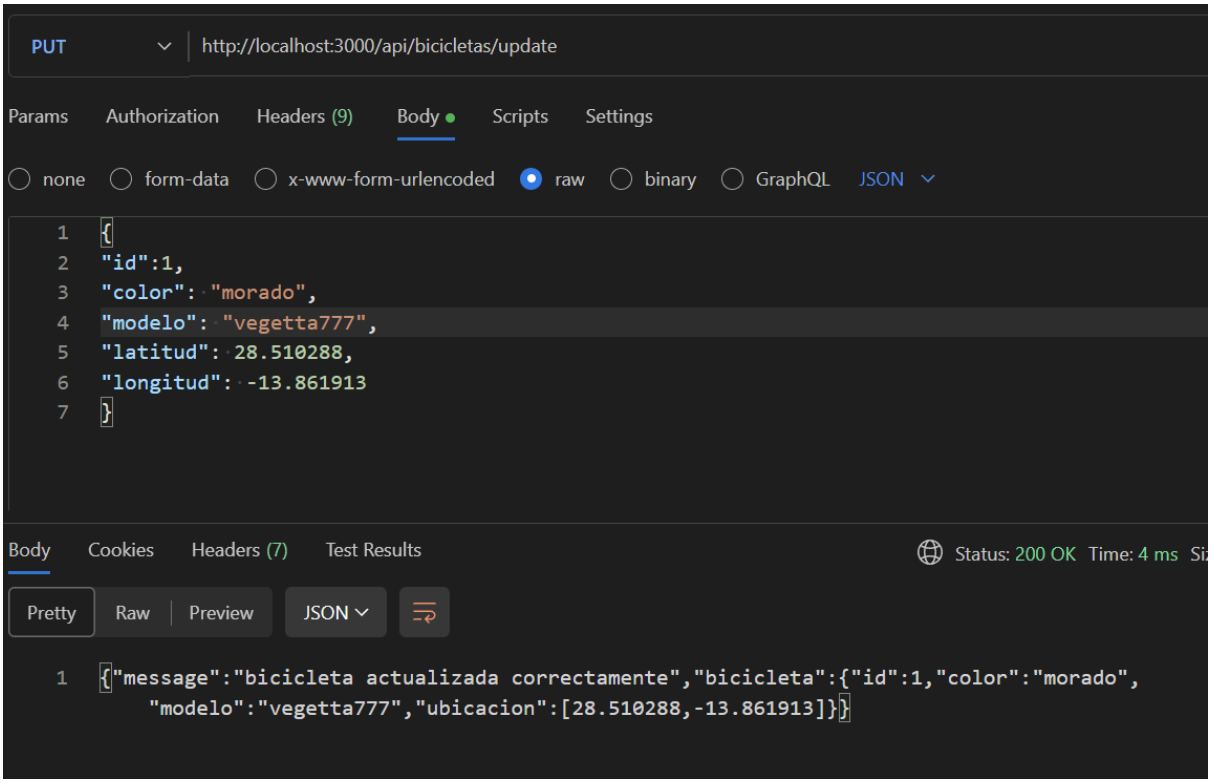
```
1
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

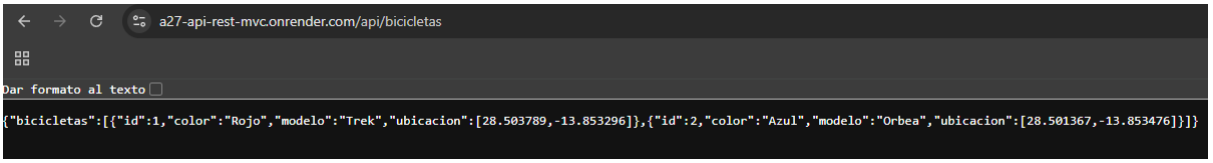
```
PS C:\Users\Arthur\Documents\Informatica\DPL\A27. API REST\API-REST> nodemon ./bin/www
[nodemon] starting `node ./bin/www`
GET /api/bicicletas 500 185.104 ms - 1582
[nodemon] restarting due to changes...
[nodemon] starting `node ./bin/www`
GET /api/bicicletas 200 7.670 ms - 167
GET /api/bicicletas/create 404 178.656 ms - 1458
POST /api/bicicletas/create 201 0.722 ms - 90
[nodemon] restarting due to changes...
[nodemon] starting `node ./bin/www`
[nodemon] restarting due to changes...
[nodemon] starting `node ./bin/www`
[nodemon] restarting due to changes...
[nodemon] starting `node ./bin/www`
DELETE /api/bicicletas/delete 204 7.050 ms - -
```

Ejercicio 5. PUT (update)

Crea una función update para utilizarla con el método PUT para nuevas bicicletas y pruébalo con Postman.



Realiza el despliegue en Render y prueba la API en producción.
https://a27-api-rest-mvc.onrender.com/api/bicicletas



Notas:

Incluye la URL de tu despliegue en Render en la memoria.
Adjunta a la entrega el código fuente en zip. (No incluyas la carpeta node_modules)